# 22.SWITCH TRANSFORMERS

24 March 2026 19:13

Models typically reuse the same parameters for inputs

Mixture of Experts defy this and instead select different parameters for each incoming example

A sparsely activated model
→ Outrageous no of parameters
↳ a constant computational cost

→ It's not just different parameters
Only a subset participates in forward + backward pass
Thus affects compute, gradient flow

→ Mechanism Parameters ↑↑
Active parameters per token ≈ constant
So
FLOPs per token ≈ constant
Model Capacity ↑

MoE
Limitations → complexity
Communication costs
training instability
] Address with Switch transformer

→ in dense transformers computation is mostly local
In sparse / MoE
Computation may involve moving data across devices

This paper simplifies MoE routing algorithm, designs models with reduced communication & computation costs

Shows large sparse models may be trained with lower precision formats

distillation here
Teacher Sparse large model
Student dense small model

Meaning Sparse model learns something transferable
Representation lens
Very large parameter space
Only small subspace used per input

* This paper is pre-chinchilla (Hoffmann et al)

Scale → yes Need for computational efficiency — they propose a sparsely-activated expert model The Switch Transformer

Sparsity comes from activating a subset of neural network weights for each incoming example

Large sparse models can be distilled into small dense versions preserving 30% of the sparse model quality gain

Switch Transformer Architecture Simplifies and improves over Mixture of Experts

Scaling properties and a benchmark against the strongly tuned T5 model

Successful distillation of sparse pre-trained and specialized fine tuned models into small dense models

Improved pretraining & fine tuning techniques

(i) Selective precision training that enables training with lower bfloat16 precision
(ii) An initialization scheme that allows for scaling to a large no of experts
(iii) Increased expert regularization that improves sparse model fine tuning and multi task training

Measurement of pretraining benefits on multilingual data with universal improvement

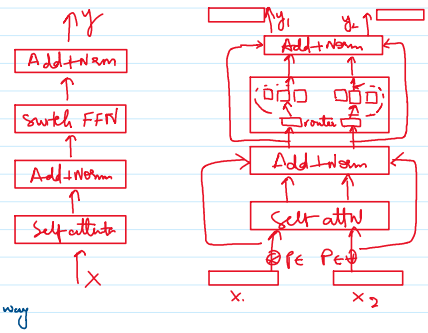
An increase in the scale of neural language models achieved by efficiently combining data, model & expert parallelism to create models with upto a trillion parameters

Contributions of this

Remove sparsity
Compute ∝ parameters

Information flow
Input → Params subset → output

Not all params participate
Gradients only update active subset




Architecture

* To maximize the parameter count of a transformer model in a simple and computationally efficient way

→ Increase the param count while keeping FLOPs constant per example

High level objective Maximize parameters under compute constraint

Connect sparsity to efficient use of dense hardware

Distributed training setup

Sparsely activated layers split unique weights on different devices
↳ The weights of model increase with no of devices, all while maintaining a manageable memory and computational footprint on each device

Distributed nature across devices → parameters scale with devices
memory per device stays manageable

Mixture of experts Routing —

MoE routing Mechanism —

Router produces logits → softmax → probabilities

Output → weighted sum of expert outputs

input a token representation $x$
routes this to the best determined top K experts, selected from a set $\{E_i(x)\}_{i=1}^{N}$ of N experts

The router variable $W_r$ produces logits $h(x) = W_r x$ → normalized via softmax distn over the available N experts at that layer

The gate value for expert $i$, $P_i(x) = \frac{e^{h(x)_i}}{\sum_j^N e^{h(x)_j}}$

$p_i(x)$ is differentiable
But routing decision (top K) introduces non smoothness

Top K gate values are selected for routing the token $x$

If $\mathcal{T}$ is the set of selected top K indices then the output computation of the layer is the linearly weighted combination of each experts computation on the token by the gate value

$$y = \sum_{i \in \mathcal{T}} P_i(x) E_i(x)$$

K = 1 (switch routing) → reduces compute
↓ Simplified routing
↓ reduced communication

Dropped tokens Overflow → skip + residual path

What happens if one expert gets more tokens?

Switch Routing

$$y = \sum_{i \in T} P(x) E_i(x)$$

## Switch Routing

2017 { Routing to $k>1$ experts was necessary in order to have non trivial gradients to routing functions
Learning to route would not work without atleast two experts

2018 → Higher k values in lower layers of the model were important for models with many routing layers

What happens if one expert gets more tokens?
→ the loss is not just about token count
$P_i$ is differentiable (softmax) $f_i$ is discrete
↓ depends on router weights gradient flows
↓ depends on argmax routing decision no gradient

This paper → route to a single expert
this preserves model quality, reduces routing computation & perform better

K=1 routing strategy is later referred to as a switch layer
→ Routing computation is reduced (routing only to one expert)
→ Batch size (expert capacity) of each expert can atleast be halved since each token is routed to a single expert
→ Routing implementation simplified → communication costs reduced

Expert capacity = Batch size of each expert

$$= \frac{\text{tokens per batch}}{\text{no of experts}} \times \text{Capacity factor}$$

loss $= \alpha N \sum f_i P_i$
$f_i$ = actual load (discrete)
$P_i$ = router belief (continuous)
Alignment between where tokens go and where token puts probability

## Distributed switch implementation

tensor shapes are statically determined at compilation time
computation is dynamic due to the routing decisions at training and inference

How to set the expert capacity?
↓
no of tokens each expert computes

capacity factor > 1.0 creates additional buffer to accommodate for when tokens are not perfectly balanced across experts

If too many tokens are routed to an expert (dropped tokens)
↳ computation is skipped & token representations are passed to next layer through residual connection

Increase the expert capacity too much → waste compute and memory

Important to ensure lower rates of dropped tokens for scaling sparse-expert models

No dependency b/w no of experts & no of tokens dropped

Uniform Case (if perfectly balanced)
$f_i = \frac{1}{N}$ $P_i = \frac{1}{N}$
then $\sum f_i P_i = \frac{1}{N}$

Non uniform case
If one expert dominates
$f_k \uparrow$ $P_k \uparrow$ $\sum f_i P_i \uparrow$
Loss increases

The loss pushes
Don't concentrate probability mass where tokens already concentrate

Adjust router probabilities $P_i$ to spread load
→ Loss couples actual routing ($f$) with router belief ($P$)
and gradients act only through $P$

The only configuration where no expert is penalized no f.i.P. dominates is $f_i = P_i = \frac{1}{N}$

→ for overloaded experts reduce $P_k$ → for underused experts increase $P_i$

→ if expert k is overloaded
$f_k \uparrow$
loss pushes → decrease $P_k$
Since P must sum to 1
other $P_i \uparrow$
This forces routing away from overloaded expert

## A differentiable load balancing loss -

Auxiliary loss to ensure a balanced load across experts

For each switch layer, this auxiliary loss is added to the total model loss during training

N experts $i = 1$ to $N$ batch B T tokens

$$\text{loss} = \alpha N \sum_{i=1}^{N} f_i P_i$$

$\alpha$: hyperparameter
$P_i$ → fraction of router probability allocated for expert i
$f_i$ → fraction of tokens dispatched to expert i

$$f_i = \frac{1}{T} \sum_{x \in B} \mathbb{1}\{\text{argmax } p(x) = i\}$$ → not differentiable

$$P_i = \frac{1}{T} \sum_{x \in B} P_i(x)$$ → differentiable

We seek uniform routing of the batch of tokens across N experts, we desire both vectors to have values of $1/N$

final loss is multiplied by expert count N to keep the loss constant as the no of experts varies since under uniform routing

$$\sum_{i=1}^{N} (f_i P_i) = \sum_{i=1}^{N} \left(\frac{1}{N} \frac{1}{N}\right) = \frac{1}{N}$$

$\alpha = 10^{-2}$ large enough ensure load balancing
small enough not to overwhelm the primary cross entropy objective

→ Switch transformers outperform both carefully tuned dense models & MoE Transformers on a speed-quality basis for a fixed amount of computation & wall clock time

Gradients → Selected expert gets gradients
Token → chosen expert
Loss → backprop → that expert updates

Non-selected experts do not get gradients
↳ They are completely inactive for that token

Sparsity/MoE → Sparse activation = Sparse gradients updates

Top 1 Selection (argmax) → non differentiable
So gradient doesn't flow through discrete choice

How does router learn?
Soft max probabilities (before argmax)
auxiliary loss
Router is trained by probabilities not by discrete routing decision

→ Switch transformers outperform both carefully tuned dense models & MoE Transformers on a speed-quality basis for a fixed amount of computation & wall clock time

→ Switch transformer has smaller computational footprint than MoE

if we increase the size to match MoE training speed, then outperforms all MoE & dense models on a per step basis as well

→ Switch transformers perform better at low capacity factors (1 & 1.25)

Small expert capacities are indicative of the scenario in the large model regime where model memory is very scarce & capacity factor will want to be made as small as possible

Selective precision with large sparse models

{ Selectively casting to float 32 precision within a localized part of the model, stability can be achieved without increasing the expensive communication cost of float 32 tensors

Mixed precision training strategy

→ cast router input to float 32 precision
→ Router function turns tokens as input and produces the dispatch & combine tensors used for selection and recombination of expert computation

Float 32 precision is only used within the body of router function on computations local to that device

Because the resulting dispatch & combine tensors are recast to bfloat 16 precision at end of function no expensive f64 & f32 tensors are broadcast through all to all communication operations but we still benefit from f32 operations

Small parameter initialization for stability

Initialize wt matrices by drawing elements from a truncated normal distn $\mu = 0$ $\sigma = \sqrt{s/n}$ → Scale hyper parameter, → no of inputs in weight tensor

Regularizing large sparse Models -

Switch transformers have significantly more parameters than FLOP matched dense baseline

↳ this may lead to severe overfitting on these smaller downstream tasks

↳ Increase the dropout rate inside the experts {expert dropout}

During fine tuning we simply increase the dropout rate by a significant amount only at the interim feed forward computation at each expert layer

Simply increasing dropout across all layers leads to worse performance

Information flow -

1. Transformer flow (pre router)
   embedding + positional
   self attention
   residual + norm
2. Replacing FFN with router + experts
   FFN → MoE (router + experts)
3. Routing step
   router computes logits → selects top 1 expert
4. Expert computation
   tokens goes to one expert (FFN like)
   expert processes it
5. Output combination
   output multiplied with gate value
   gate ≈ scalar weight
6. Residual path
   input bypasses expert via residual

→ Ot

Router takes → token representation x
There are a two independent gradient paths

Main loss (task loss)
flows
token → expert → output → loss
Updates
Selected expert
router (via P(x), indirectly)

Auxiliary loss
Computed from
→ routing statistics f, P
flows directly into router parameters

Where decision happens
↳ Router (argmax over P(x))

→ Forward pass
Router computes P(x)
Argmax picks expert k

→ Backward pass
Expert k gets gradients
Router gets gradients via P(x) not via argmax path

Consequences
experts
→ sparse, uneven updates
Router
→ dense signal (via probabilities + aux loss)

STABILITY -
↳ uneven gradient updates across experts
↳ Routing skew → some experts overloaded
↳ Capacity/dropped tokens
↳ overflow → tokens skipped
↳ information loss + inconsistent learning signal
→ Activation variability → different experts
↳ diff activation distb's

These effects amplify at scale

CORE INSTABILITY MECHANISM

1. Positive feedback loop
   slight imbalance → one expert gets more tokens
   gets more tokens
   becomes better
   router prefers it more
   Collapse toward few routers
2. Gradient variance explosion
   Some experts → high freq updates
   Others → almost none
   Non uniform learning dynamics
3. Router expert mismatch
   Router probabilities (soft)
   Actual routing (hard)
   Unstable optimization signal
4. Dropped tokens → broken signal
   no expert processing → only residual path
   inconsistent gradient flow

Feedback driven imbalance + sparse updates + training collapse risk

→ Scaling via no of experts
more experts → more parameters
Active compute → stays constant

→ Extra routing cost
latent cost d_model × no of experts
router cost
↳ smaller relative to FFN compute

→ step efficiency ≠ wall clock efficiency
Communication overhead matters

→ Sparse v dense → sparse wins under compute constraint

Expert + data parallelism

1. What problem are they solving?

6 Residual path
input bypasses expert via residual

↳ Router (argmax over P(x))

→ Router compresses routing decision
↳ restricts computation path

Scaling -
Router must compute a probability distribution over more experts
even if it has to select only one expert

light wt computation cost $O(d_{model} \times$ no of experts)

Step basis -
Keeping FLOPs per token fixed, having more params (experts) speeds up training

Increasing no of experts leads to more sample efficient models
learning more quickly for a fixed no of tokens

Time basis :-
Switch transformers → communication costs across devices and extra computation for routing mechanism

Increased sample efficiency on step basis doesn't necessarily translate to a better model quality as measured by clock

? for a fixed duration & computational budget, should one train a dense or a sparse model

? what if we allocate resources to a large dense model?

Distillation -
Deploying massive neural nets with billions/trillions of params inconvenient
distill large models into small ones

Arbitrarily increasing no of experts is subject to diminishing returns

(SPMD) → single program multiple data parallelism

B tokens
$d_{model}$ each token dimension
x input
y output } $[B, d_{model}]$
h → intermediate $[B, d_{ff}]$ → several times larger than $d_{model}$

$h = x W_{in}$ $[d_{model}, d_{ff}]$ $W_{in}$, $W_{out}$ } applied independently on each token
$y = ReLU(h) W_{out}$ $[d_{ff}, d_{model}]$

N → No of total cores $[N = m \times n]$
n → no of ways for data parallelism sharing
m → no of ways for model parallelism sharing

E → no of experts in switch layers
C → expert capacity, the batch size of each expert

To shard layer across cores
↳ the tensors containing batch B tokens are sharded across n-data parallel cores, so each contains B/n tokens
↳ Tensors & variables with $d_{ff}$ are then sharded across m model parallel cores

Data parallelism
all cores are allocated to data-parallel dimension $n=N, m=1$
No communication needed until entire forward and backward pass is finished
and the gradients needed to be then aggregated across all cores

Model parallelism
$n=1$ $m=N$
all cores must keep full B tokens and each core will contain unique slice of weights
for each forward & backward pass → a communication cost occurs
Each core sends a tensor $[B, d_{model}]$ to compute second matrix mult $ReLU(h(W_{out}))$
because $d_{ff}$ dimension is partitioned and must be summed over
A general rule → whenever a dimension that is partitioned across cores must be summed

① What problem are they solving?
Constraint Experts on different devices
Tokens must go to their assigned expert
tokens must be moved across devices

② What happens per batch?
B = Batch size E = experts
Cores = distributed

③ Routing output
Router produces
for each token assigned expert

④ Constructing Routing Matrix
They build [B,E] binary assignment matrix
1 → goes to expert
0 → otherwise

⑤ partition across devices
Each device
owns one expert

⑥ Core operation
einsum → gather tokens per expert
effect
tokens are re-grouped by expert

⑦ New shape becomes
$[B, E, C, d_{model}]$
each expert gets its tokens
capacity = C

⑧ Communication
all to all communication
tokens move across devices
each expert receives its assigned tokens

⑨ After expert communication
Reverse happens
↳ outputs sent back
↳ recombined into original order

3D partitioning problem → data, → model, → experts

Expert parallelism
Send tokens to the device that owns their expert, processes them, then sends results back

Constraint → memory per device limited
→ experts distributed

Information flow →
token → router → device transfer → expert → return

Scaling →
more experts → more device
more communication

→ Increasing experts → sample efficiency?
Yes different experts specialize → better token handling

Routing enables specialization

With dense model
All params used by every token
capacity shared globally

With sparse model

Each core [B/n, d model] to compute [illegible]
because dff dimension is partitioned and must be summed over

A general rule → whenever a dimension that is partitioned across cores must be summed then an all-reduce operation is added for both the forward & backward pass

In data parallelism this occurs only at the end of entire forward & backward pass

## Model & data parallelism

N = n×m cores

each core responsible for $\frac{B}{n}$ tokens & $d_{ff}/m$ of both weights & intermediate activation

In forward & backward pass each core communicates a tensor of size [B/n, d model] in an all-reduce operation

## Expert & data parallelism

allocate all cores to data partitioning dimension n (corresponds to no of experts)

For each token per core a router locally computes assignments to the experts

The output is a binary matrix of size [n, B/n, E, C] which is partitioned across first dimension & determines expert assignment

used to do a gather with input tensor of [n, B/n, d model]

einsum([n, B/n, d model], [n, B/n, E, C], dimension = [B/n])

final tensor shape [n, E, C, d model] sharded across first dim

Because each core has its own expert, we do an all to all communication of size [E, C, d model] to now shard the E dim instead of n dim

There are additional communication costs of bf16 tensors of size E×C×d model in the forward pass to analogously receive the tokens from each expert located on different cores

## Expert Model & Data parallelism

aim → balance b/w FLOPs per token & parameter count

When we scale experts → we scale params → but FLOPs don't change

to increase FLOPs → increase dff (increases params but at slower rate)

trade off dff → runs out of memory per core which necessitates increasing m ↑

So N is fixed N = n×m hence decrease n!

this forces use of a smaller batch size (in order to hold tokens per core constant)

All params used by every token
capacity shared globally

With switch transformer
Total params ↑
But per-token compute = const

Why this improves efficiency?
→ Reduced interference
Dense model
all tokens update same params
gradients interfere
Sparse model
diff tokens → diff experts
less gradient interference

→ Higher effective capacity per bit
Compute const, params ↑
routing selects relevant subset
each token sees a better matched function

→ Faster learning signal
→ Experts specialize quickly
→ gradients are more consistent within expert
leads to fewer samples needed to learn patterns

* Reduced gradient interference
+ Specialized parameter subspaces